TWO H2O ポートフォリオ



東京工芸大学 芸術学部
ゲーム学科 プログラム分野 4年
大野 栞里

目次

1. 概要
2. 担当箇所
3. 担当ソースコード
4. 工夫点

1. 概要

- ジャンル
  ２Dアクション協力ゲーム

- 開発目的
  コウゲイゲームショウへの展示

- 開発期間
  2024年 6月17日 ～ 2025年 1月14日（6ヵ月28日）

- チーム構成
  企画 2名 デザイン2名 プログラム 4名（計8名）

1. 概要

- 開発環境
  Unity2022.3.25f1
  Visual Studio 2022
  C#
  GitHub
  GitHub Desktop
  Visual Studio Code

- GitHub URL
  プロジェクトファイル

2.担当箇所

- プレイヤー
  プレイヤーの向き
  物理演算（一部）

- ギミック
  プロペラ
  ボタン
  水
  リスポーン地点

- アウトゲーム
  タイトル画面
  クリア画面

3. 担当ソースコード

- 目次
  - GameplayButtonsフォルダ
  - Managersフォルダ
  - Playerフォルダ
  - Propellerフォルダ
  - RespawnStateフォルダ
  - UIフォルダ

3. 担当ソースコード

- GameplayButtonsフォルダ

ファイル名	概要
BaseButton.cs	基本的なプレイヤーのキャラクターが押下するボタンの処理を抽象化したクラス。
DisappearPuddle.cs	ボタン押下で水たまりを非表示にするクラス。
IButtonAction.cs	ボタンアクションのインターフェイス。
RepressableButton.cs	ボタンを押下することが何度でも可能な基底クラス。
ReverseRotationButton.cs	ボタン押下でプロペラの回転方向を反転させるクラス。
StartRotateButton.cs	ボタン押下でプロペラの回転を始めるクラス。
WaterLevelChangeButton.cs	ボタン押下で水位の移動を非同期で実行するクラス。

3. 担当ソースコード

- Managersフォルダ

ファイル名	概要
GameManager.cs	シーン遷移と終了を管理しているクラス。
goalDecisionScript.cs	ゴール判定をした後のシーン遷移をしているクラス。

3. 担当ソースコード

- Playerフォルダ

ファイル名	概要
DirectionSwap.cs	入力でプレイヤーキャラクターの向きの変更をするクラス。
ice_above_water.cs	氷状態のキャラクターが水上で動くクラス。
puddle_Script.cs	水状態のキャラクターが水上に接触している時間を管理しているクラス。

3. 担当ソースコード

- Propellerフォルダ

ファイル名	概要
GasDirection.cs	プロペラの範囲内に入った時、ガス状態のキャラクターの飛行する方向を管理するクラス。
RiseGas.cs	プロペラの範囲内に入った時、ガス状態のキャラクターの飛行するスピードの調整をするクラス。
RotateObject.cs	プロペラの回転を制御しているクラス。
changeWhereMoveTag.cs	プロペラの回転でタグ切替を管理するクラス。

3. 担当ソースコード

- RespawnStateフォルダ

ファイル名	概要
EyeBlendShapeController.cs	アニメーション完了時に、目を開く制御をするクラス。
TriggerAnimationMask.cs	リスポーン更新地点で、トリガー検知でアニメーションを再生するクラス。

3. 担当ソースコード

- UIフォルダ

ファイル名	概要
BackImageButton.cs	Backに設定したボタンで操作説明の画像を戻すクラス。
ButtonScaleScript.cs	選択時にUIのボタンの拡大縮小をするクラス。
ImageSwitcher.cs	UIの操作説明のボタンを押した時の処理と、画像の順次切替を制御するクラス。
InputHandler.cs	Unity Input System を活用してNextImageアクションを検知し、それに応じた処理を実行するためのクラス。
canvasAppear.cs	ゴールした時のUI表示をするクラス。
titleSceneTransition.cs	任意のキーで操作説明の画像を次に進めるクラス。

4. 工夫点

- プレイヤーキャラクターが押すボタンの設計

  共通処理の抽象化

  ボタンの共通機能をBaseButtonやRepressableButtonなどの抽象クラスに集約した。

  共通ロジックを一箇所にまとめ、メンテナンス性が向上するようにした。

  拡張性の向上

  IButtonActionインターフェースを利用し、各ボタンの具体的なアクションを定義した。

  新しいボタンについて、インターフェースを実装した具象クラスとして簡単に追加が可能になるようにした。

4. 工夫点

- 非同期処理の活用

  UniTaskを使うことで効率的に操作管理出来るようにした

    ボタンアクションなどに非同期処理を導入し、スムーズな操作を実現した。